

Student Declaration of Authorship



Course code and name:	F20SC – Industrial Programming
Type of assessment:	Individual
Coursework Title:	Coursework 2 – Data Analysis of a Document Tracker
Student Name:	Archisa Kar, Asmitha Krishnakumar, Suhail Ahmed
Student ID Number:	H00363145, H00376043, H00384341

Declaration of authorship. By signing this form:

- **I declare** that the work I have submitted for individual assessment OR the work I have contributed to a group assessment, is entirely my own. I have NOT taken the ideas, writings or inventions of another person and used these as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements section.
- I confirm that I have read, understood and followed the University's Regulations on plagiarism as published on the [University's website](#), and that I am aware of the penalties that I will face should I not adhere to the University Regulations.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on [Academic Integrity and Plagiarism](#)

Student Signature (type your name): Archisa Kar, Asmitha Krishnakumar, Suhail Ahmed

Date: 05/12/2023

Table of Contents

1. Introduction	4
2. Requirements Checklist	4
3. Design Considerations	4
3.1 Code Structure	4
3.2 Data Structures	5
3.3 Scalability	5
3.4 Exception Handling	5
4. User Guide	5
4.1 Executing in Command Line.....	5
4.2 Running GUI.....	6
5. Developer Guide.....	10
5.1 Code Organization.....	10
5.2 Libraries Used	11
5.3 Entry Point of the Application	11
5.4 Execution of the code.....	11
5.4.1. Command Line Interface	12
5.4.2. Graphical User Interface.....	12
5.5 Core Functionalities	12
5.5.1. Reading Data.....	12
5.5.2. Country and Continent Analysis.....	13
5.5.3. Browser Analysis	13
5.5.4. Visitor Engagement Analysis	13
5.5.5. Also Likes	14
5.6 Additional Features	14
5.6.1. Identifying documents with the highest unique visitor count	14
5.6.2. Also Likes Testing	14
5.6.3. Sorting Function	14
5.6.4. Checking Documents with Multiple Countries	15
5.6.5. Validation Feature	15
5.6.6. Analyzing Popular Reading Time for Documents.....	15
5.6.7. Analyzing Popular Reading Time for Visitors.....	15
5.6.8. Categorizing User Locations based on IP	15
5.6.9. Counting number of logged-in and non-logged-in visitors based on their source	15
5.6.10. Identifying Authenticated Visitors	15
5.7 Graphs and GUI.....	16
6. Testing	16
6.1 Invalid Input.....	16
6.2 Authenticating Inputs.....	16

6.3 Verifying Country and Continent Count	17
6.4 Avid Reader Functionality	17
6.5 `Also_likes` Functionality	17
7. What did I learn from CW1?	18
7.1 Code Feedback	18
7.2 Report Feedback	18
8. Conclusions	18
8.1 Future Works	18
8.2 Comparison of Software Development in Scripting vs System Language	18
9. References	19
10. Appendix.....	19
10.1 Contributions	19

1. Introduction

The purpose of this report is to document the design, implementation, and testing of a Python-based application. The application is used for the analysis and display of document tracking data sourced from issuu.com. Developed on VSCode, the application underwent testing on a MacBook Air M1, a MacBook Pro M1, and an Acer Spin 3. It is designed to run across all platforms.

The following assumptions were made during the development process:

- The documentation was written assuming that the readers have prior knowledge of Python and programming concepts.
- It is assumed that the timestamps provided in the JSON file follow a universal format.
- The dataset used for analysis is free from noise or inconsistencies.

2. Requirements Checklist

Requirements	Status
R1 : Views by country/ continent	Completed
R2 : Views by browser	Completed
R3 : Reader Profile	Completed
R4 : “Also likes” functionality	Completed
R5 : “Also likes” graph	Completed
R6 : Develop GUI	Completed
R7 : Command Line Interface	Completed
R8 : Testing	Completed
R9 : Additional Features	Completed

3. Design Considerations

3.1 Code Structure

The code adopts a structured, object-oriented approach, organizing functions into modules for clarity. Unit tests are integrated. **cw2.py** is the entry point, offering a GUI or CLI choice with **argparse** for CLI handling. Tests in **unit_test.py** use **PyTest** to evaluate core functionalities. Modules like **cw_requirements.py**, **additional.py**, **graphs.py**, and **gui.py** handle tasks regarding analyses, graphs and GUI ensuring a concise and organized codebase. The **gui.py** module has multiple classes contributing to different aspects of the GUI, promoting modularity in the overall code structure.

3.2 Data Structures

The code predominantly operates on data structures such as lists and dictionaries. The core data entities, namely documents and visitors, are stored as dictionaries. The documents are structured with universal unique identifiers (UUID) as keys, and each document contains relevant information. Similarly, the visitors are organized with their UUIDs as keys, facilitating efficient retrieval of visitor information. For storing the JSON data, a list was used to maintain a sequential collection of JSON objects from the file and for dynamic storage. Throughout the code, these data structures are manipulated and queried to perform various tasks, for the functionalities implemented.

3.3 Scalability

The program is designed to handle different tasks related to data analysis, allowing users to interact with the data through both, the Graphical User Interface (GUI) and Command Line Interface (CLI). The use of modular functions and libraries facilitate scalability, enabling the addition of new functionalities without major code restructuring. Furthermore, the code's scalability is proven by its capacity to process datasets ranging from 100,000 to 3 million lines, even with significant increase in data volume.

3.4 Exception Handling

Error handling is implemented to ensure that the program handles unexpected scenarios and prevents crashes while running the application. Unit tests such as file path and UUID validation test the robustness of the code. In the event of any corner case, an exception is raised informing the user of the same.

4. User Guide

4.1 Executing in Command Line

The following code is used to execute the application from the command line:

```
` python cw2.py -u <user_uuid> -d <doc_uuid> -t <task ID> -f <file_name.json> `
```

Here's a breakdown of the options:

<user_uuid> : Specify the visitor UUID.

<doc_uuid> : Specify the document UUID.

<task ID>: Specify the task ID.

<file_name.json> : Include a file name in JSON format.

Example Usage:

```
asmithakrishnakumar@Asmithas-MacBook-Air DataAnalysis % python3 cw2.py -u aaa4eaf77abab0b2 -d 130323125939-5f4318404cda4025a2463c66435ad7c8 -t 6f -f Dataset/sample_small.json
```

```
Visitor UUID: aaa4eaf77abab0b2
```

```
The visitor is not logged in.
```

If the user provides an invalid input or fails to specify any required input, the application will raise an exception.

4.2 Running GUI

To launch the GUI, run the command `python cw2.py` in the terminal without including any command line arguments. Upon executing the command, users will be prompted to choose a dataset from their system using file explorer, and following that, the GUI tab will be displayed.

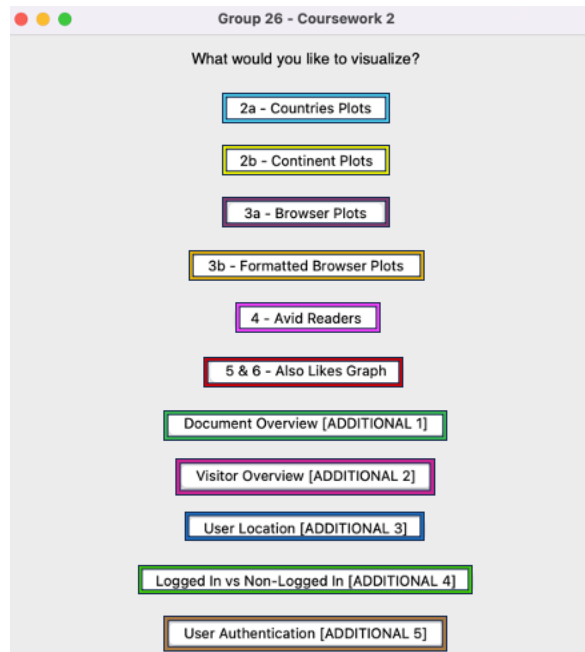


Figure 1: Homepage

Plotting graphs for number of views from each country (Highlighted in Sky blue)

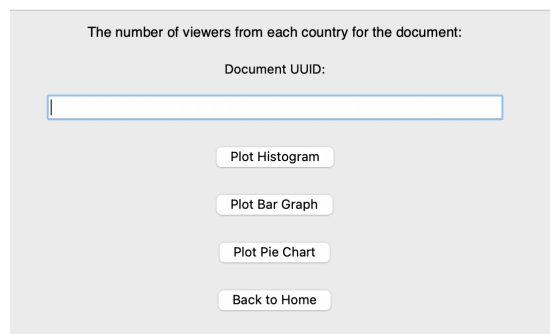


Figure 2: New Tab for Countries Plot

When the **Countries Plots** button is clicked, a new tab will appear, prompting the user to input the document UUID and choose a graph type such as Histogram, Bar Graph (**Additional Feature**) or Pie Chart (**Additional Feature**). The **Back to Home** button will redirect the user to the homepage.

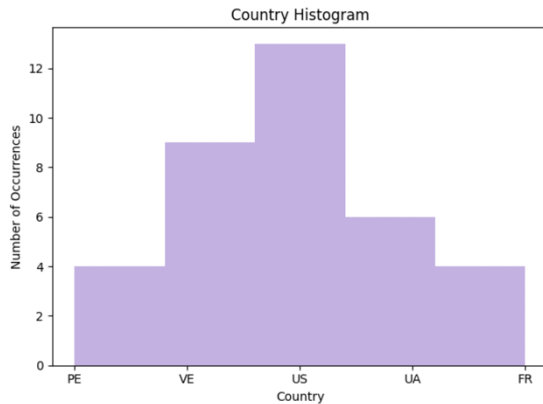


Figure 3: Output for histogram

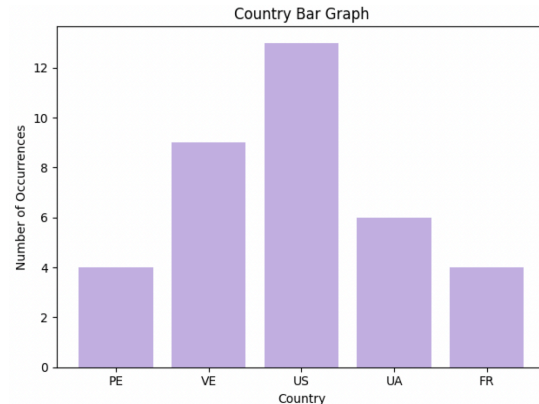


Figure 4: Output for Bar Graph

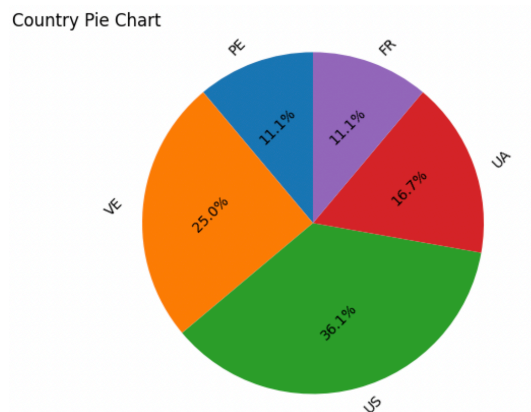


Figure 5: Output for Pie Chart

Plotting graphs for number of views from each continent (Highlighted in Yellow)

When the **Continent Plots** button is clicked, a new tab will appear [Image 1], prompting the user to input the document UUID and choose a graph type such as Histogram, Bar Graph (Additional Feature) or Pie Chart (Additional Feature). The Back to Home button will redirect the user to the homepage.

Plotting graphs for number of views from different browsers (Highlighted in Purple)

Upon clicking the **Browsers Plots** button, a new tab will appear [Image 2], offering users the choice to choose either Histogram or Bar Graph (Additional Feature). After selecting the graph type, the application will generate and display the chosen graph. The **Back to Home** button will redirect the user to homepage.

Plotting formatted graphs for the number of viewers from different main browser (Highlighted in Orange)

When clicking the **Formatted Browser Plot** button, a new tab will appear, offering users the choice to choose a graph type such as Histogram, Bar graph, and Pie Chart. Upon selecting a graph type, the application will generate and display the chosen graph. The **Back to Home** button will redirect the user back to the home page.

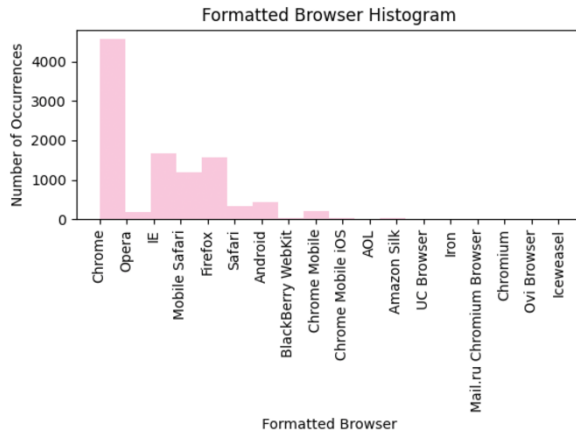


Figure 6: Formatted Browser Histogram

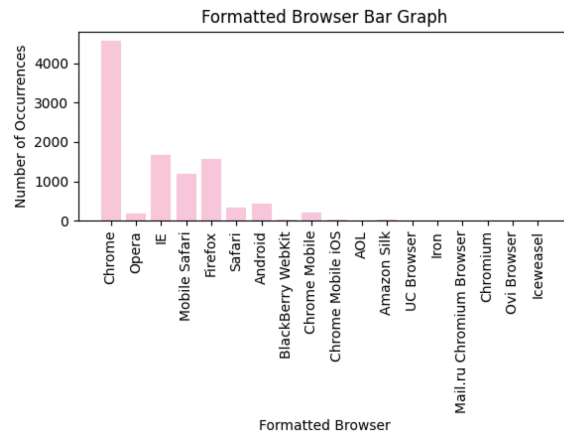


Figure 7: Formatted Browser Bar Graph

Plotting graph for top 10 avid reader (Highlighted in Magenta)

On clicking the **Avid Readers** button, a new tab will appear, prompting the user to enter the document UUID. The application will generate a bar graph for top 10 avid readers of a document along with their UUID and reading time [Image 3]. The **Back to Home** button will redirect the user to the homepage.

Also Likes Graph (Highlighted in Red)

When the **Also Likes Graph** button is clicked, a new tab will appear [Image 4], prompting the user to input the document UUID, Visitor UUID (Optional), Sort Order (Optional). The Sort Order parameter allows users to specify the desired sorting function for the generated graph. If the Sort Order is left unspecified, the application will automatically return the graph in descending order. The application will generate a graph for **also_likes** functionality and will also display information pertaining to it in the GUI. The **Back to Home** button will redirect the user to the homepage.

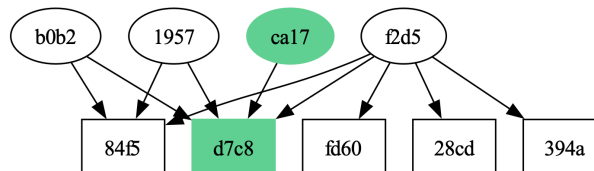


Figure 8: Also Likes Graph

Plotting graph for Document Overview (Highlighted in Green)

Upon clicking the **Document Overview** button, a new tab will appear, prompting the user to enter the document UUID (Optional). The application will generate a graph for the most common time of the day when a specified document UUID is read. If the document UUID is not specified, the function returns the most common time of the day when all documents are read. The **Back to Home** button will redirect the user to the homepage.

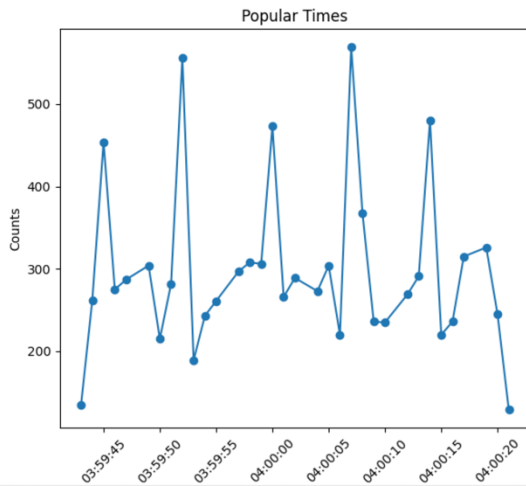


Figure 9: Output when document UUID is not specified.

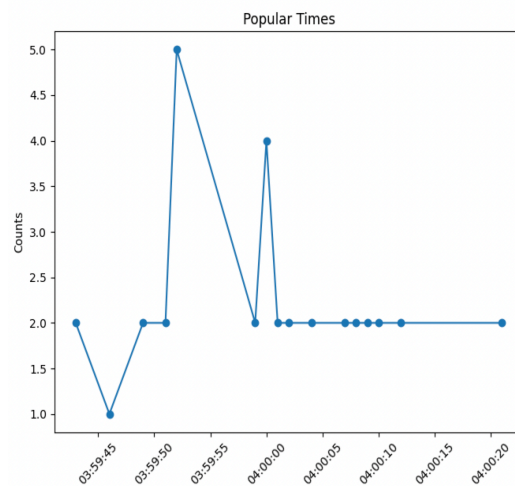


Figure 10: Output when document UUID is specified.

Plotting graph for Visitor Overview (Highlighted in Fuchsia)

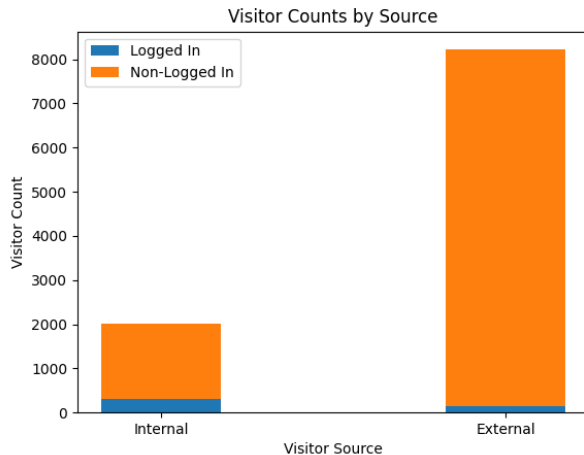
Upon clicking the **Visitor Overview** button, a new tab will appear [Image 5], prompting the user to enter the Visitor UUID. The application will generate a graph for the most common time of the day when a specified visitor reads the documents. If document UUID is not specified, the function returns the most common time of the day when all documents are read. The **Back to Home** button will redirect the user to homepage. [Image 6]

Plotting graph for User Pseudo-Labels (Highlighted in Dark Blue)

When **User Location** button is clicked, a new tab will appear that has two buttons: **Plot** and **Back to Home**. On clicking **Plot** button, a pie chart for the user location is generated, [Image 7]. The pie chart generated uses pseudo-labeling to categorize events between 9 A.M. to 5 P.M. as office/school views and others as home views.

Sources for Logged-in vs Non-logged-in Users (Highlighted in Light Green)

When **Logged In Users** button is clicked, a new tab will appear that has two buttons: **Plot** and **Back to Home**. On clicking the **Plot** button, a bar chart is generated for both logged in and logged out users along with their source. The grouped bar chart below illustrates visitor counts categorized by source. In the **Internal** bar with a total of 2000 visitors, fewer than 500 are logged in while the majority are non-logged-in users. The **External** bar displays a total of 8000 visitors, with only a small fraction being logged-in users and majority of the users are not logged in.



Total Number of Visitors: 1329

Number of Logged In Users: 447

Source	Count
Internal	305
External	142

Number of Non-Logged In Users: 9793

Source	Count
External	8079
Internal	1714

Figure 11: Output for Logged In Users.

User Authentication (Highlighted in Brown)

On clicking the **User Authentication** button, a new tab will appear, prompting the user to enter the Visitor UUID. The application will check if the user is logged in or not and displays the status along with visitor UUID [Image 8]. The **Back to Home** button will redirect the user to the homepage.

5. Developer Guide

5.1 Code Organization

```

DataAnalysis
├── Dataset
│   └── sample_small.json
├── Functions
│   ├── __init__.py
│   ├── __pycache__
│   │   ├── __init__.cpython-311.pyc
│   │   ├── additional.cpython-311.pyc
│   │   ├── cw_requirements.cpython-311.pyc
│   │   ├── graphs.cpython-311.pyc
│   │   └── gui.cpython-311.pyc
│   ├── additional.py
│   ├── cw_requirements.py
│   ├── graphs.py
│   └── gui.py
├── __pycache__
│   └── unit_test.cpython-311-pytest-7.4.3.pyc
├── cw2.py
├── requirements.txt
└── unit_test.py
  
```

Figure 12: Directory structure in a tree format.

Our codebase is organized within the **DataAnalysis** directory. The directory includes **unit_test.py**, **cw2.py** and two subdirectories: **Functions** and **Dataset**.

- **unit_test.py**: This file contains the unit tests that assess the core functionalities of the application ranging from exception handling, coursework requirements, additional functionalities, and the GUI.

- **cw2.py**: This file acts as the application's starting point, providing users with the option to choose between a GUI and a CLI.
- **Functions**: This subdirectory encapsulates various modules, each designed for specific functionalities
 - **__init__.py**: This file marks the **Functions** subdirectory as a Python package, allowing easy imports of functions from the contained modules [\[1\]](#).
 - **cw_requirements.py**: Core requirements specified in the coursework are implemented in this module.
 - **additional.py**: All additional features beyond the core requirements are implemented in this module.
 - **graphs.py**: This module is responsible for generating histograms, bar graphs and pie chart.
 - **gui.py**: This module contains the code for GUI functionality.
- **Dataset**: This subdirectory contains relevant datasets for analysis.

5.2 Libraries Used

- **json** is used for processing the datasets.
- **pycountry_convert** is used for mapping countries to continents.
- **ua_parser** is used to parse and extract browser names from the "visitor_useragent" field.
- **argparse** is used to handle command line parsing.
- **matplotlib** library is used to plot charts for output.
- **tkinter** library is used to build the GUI for the program.
- **graphviz** is used to display .dot format graphs.

5.3 Entry Point of the Application

`cw2.py` serves as the entry point of the application, offering users the choice between a Graphical User Interface (GUI) and a Command-Line Interface (CLI). This is performed through the following functions:

`parse_arguments()` - This function is responsible for defining the syntax for passing command-line arguments. It uses the `argparse` library to parse four command line arguments (User UUID, Document UUID, Task ID, File Name). The function checks if the user entered arguments follow the specified syntax. If the arguments are incorrect, it raises an exception for the same.

`execute_task(args)` - The `execute_task` function takes in `args` (parsed command-line argument) as an argument and performs the corresponding task based on the specified task ID. The function reads the JSON file, conducts validation checks and executes the specific function corresponding to the task ID.

5.4 Execution of the code

5.4.1. Command Line Interface

The following code is the syntax for running the program via the command-line interface.

```
`python cw2.py -u <user_uuid> -d <doc_uuid> -t <task ID> -f <file_name.json>`
```

For example, to check all the documents accessed by a specified visitor (task 5b), the following command should be executed in the terminal.

```
asmithakrishnakumar@Asmithas-MacBook-Air DataAnalysis % python3 cw2.py -u aaa4eaf77abab0b2 -d 130323125939-5f4318404cda4025a2463c66435ad7c8 -t 5b -f Dataset/sample_small.json
Visitor UUID: aaa4eaf77abab0b2
Number of Documents Read: 2

Document 1's UUID: 130323125939-5f4318404cda4025a2463c66435ad7c8
Document 2's UUID: 130428213605-d16c4651f56248559b242f75ad4884f5
```

Figure 13: Running in terminal.

5.4.2. Graphical User Interface

To launch the GUI, we should run ``python cw2.py`` in the terminal without any command-line arguments.

```
asmithakrishnakumar@Asmithas-MacBook-Air DataAnalysis % python3 cw2.py
Starting GUI...
```

Figure 14: Running GUI

5.5 Core Functionalities

``cw_requirement.py`` contains all the functions that form the backbone of the data analysis. The functions include:

5.5.1. Reading Data

``read_file(file_path)`` takes in ``file_path`` as an argument and reads the JSON file specified by the given argument. This function uses ``json`` library to parse through the JSON dataset and append the data to the list. It extracts information based on the document and visitor UUIDs from the JSON data, returning dictionaries that map document and visitor UUIDs to their corresponding JSON data. These data structures are used throughout the rest of the code for various analyses. An exception is raised if the specified file is not found, indicating that the file does not exist at the specified location.

```
def read_file(file_path):
    # Initialize dictionaries to store documents and visitors
    documents = {}
    visitors = {}

    try:
        # Open the file specified by the file_path
        with open(file_path) as f:
            # Initialize an empty list to store JSON data
            json_data = []

            # Iterate through each line in the file
            for line in f:
                # Parse the JSON data from the line and append it to the list
                json_data.append(json.loads(line))

            # Check if the JSON data contains "env_doc_id"
            if "subject_doc_id" in json_data[len(json_data)-1]:
                # Extract the document UUID and content from the JSON data
                doc_uuid = json_data[len(json_data)-1]["subject_doc_id"]
                content = json_data[len(json_data)-1]

                # Check if the document UUID is already in the documents dictionary
                if doc_uuid in documents:
                    # Append the content to the existing list
                    documents[doc_uuid].append(content)

```

```

            else:
                # Create a new list with the content for the document UUID
                documents[doc_uuid] = [content]

            # Check if the JSON data contains "visitor_uuid"
            if "visitor_uuid" in json_data[len(json_data)-1]:
                # Extract the visitor ID from the JSON data
                visitor_id = json_data[len(json_data)-1]["visitor_uuid"]

                # Check if the visitor ID is already in the visitors dictionary
                if visitor_id in visitors:
                    # Append the content to the existing list
                    visitors[visitor_id].append(content)
                else:
                    # Create a new list with the content for the visitor ID
                    visitors[visitor_id] = [content]
            return documents, visitors, json_data

    except FileNotFoundError:
        # Handle the case where the specified file path is not found
        raise Exception("File does not exist at specified location.")

```

Figure 14: Code snippet for read_file()

5.5.2. Country and Continent Analysis

The **views_country(json_data, doc_uuid)** function analyzes country occurrences in JSON data for a specified document UUID. It filters and extracts country information, returning two lists: one with a counter object (**country_count**) for occurrences and another with country names associated with document visitors (**countries**).

The **group_country(countries)** function processes the **countries** list, grouping them by continent using the **pycountry_convert** library [2]. It returns a counter object (**country_count**) with continent occurrence counts.

```

1 def group_country(countries):
2
3     # Map countries to continents
4     map_country = [pc.country_alpha2_to_continent_code(country) for country in countries]
5     # Convert continent codes to continent names
6     country_continent = [pc.convert_continent_code_to_continent_name(continent) for continent in map_country]
7     # Count occurrence of each continent
8     continent_count = Counter(country_continent)
9     return continent_count

```

Figure 15: Code snippet for group_country()

5.5.3. Browser Analysis

The **view_browser(json_data)** function analyzes JSON data, focusing on the **visitor_useragent** field, to count occurrences of each browser using a counter object. It returns **browser_count** with browser names as keys and their occurrence counts.

The **format_browser(browser_count)** function uses the **user_agent_parser** library to extract browser family information [3,4]. It updates the input **browser_count** with aggregated counts and returns the modified counter object.

5.5.4. Visitor Engagement Analysis

The **avid_readers(visitors)** function calculates total reading time for each visitor based on **event_readtime**, storing results in **count_dict**. It then sorts and returns the top 10 visitors.

The **doc_to_visitor(documents, doc_uuid)** function identifies unique visitor UUIDs for a specific document by checking **documents** for related events with a **visitor_uuid** field.

The **visitor_to_doc(documents, visitor_uuid)** function compiles unique document UUIDs read by a given visitor by iterating through the entire documents dictionary and matching events with the specified visitor UUID.

5.5.5. Also Likes

This function takes a dictionary of documents with JSON data, document UUID, optional visitor UUID, and optional sorting function. It utilizes **doc_to_visitor** to obtain a list of visitor UUIDs for the specified document and then uses **visitor_to_doc** to map visitors to visited documents, facilitating reader counting. If a sorting function is specified, the function sorts documents and selects the top 7. It counts visitor occurrences in the top documents, sorts based on the provided function, and selects the top 3 visitors. If a specific visitor UUID is provided and not in the top visitors, it's added to **visitor_top_counter**. The function establishes a mapping between top visitors and the documents they've read among the top ones, returning a nested dictionary with document UUIDs as keys and values as dictionaries indicating how many readers each document has, along with their visitor UUIDs. [\[Code Snippet 1\]](#)

5.6 Additional Features

`additional.py` contains all additional features beyond the core requirements that are specified in the coursework specification. Below is an overview of each feature:

5.6.1. Identifying documents with the highest unique visitor count

`max\_unique\_visitors(documents, visitors)` function identifies the document with highest number of unique visitors. It goes through all the documents and returns the document UUID with the highest number of unique visitors and the corresponding count.

5.6.2. Also Likes Testing

`also\_likes\_testing(documents, doc\_reader\_count)` tests the **`also\_likes`** functionality by finding records that are suitable for the function and printing them. It iterates through each document UUID in the documents dictionary, checks if the result contains more than one document and prints the document UUID.

5.6.3. Sorting Function

`sortingfunc\_test(doc\_reader\_count, reverse)` function sorts dictionaries in ascending or descending order. It is used in **`also\_likes`** function to arrange the dictionary in either ascending or descending order, depending on the specified sorting criteria. [\[Code Snippet 2\]](#)

5.6.4. Checking Documents with Multiple Countries

``find_doc(json_data, country_count)`` returns a list of document UUIDs which has visitors from multiple countries.

5.6.5. Validation Feature

``validation(doc_uuid=None, visitor_uuid=None, documents=None, visitors=None)`` function checks if document UUIDs and visitor UUIDs are valid. Exceptions are raised for invalid inputs. [\[Code Snippet 3\]](#)

5.6.6. Analyzing Popular Reading Time for Documents

``most_popular_time_documents(documents, doc_uuid=None)`` returns the most common time of the day when a specified document UUID is read. If document UUID is not specified, the function returns the most common time of the day when all documents are read.

5.6.7. Analyzing Popular Reading Time for Visitors

``most_popular_time_visitors(visited_documents, all_documents)`` function returns the most common time of the day when a specified visitor reads the documents. It iterates through each event for the document UUID, stores the timestamp for the event and increments the counter for every occurrence.

5.6.8. Categorizing User Locations based on IP

``ip_to_location(documents, ip_address=None)`` function determines the location (home/office or school) of a given IP address. The function tracks hours and categorizes events occurring between 9 A.M. to 5 P.M as office/school views and the rest as home views. [\[Code snippet 4\]](#)

5.6.9. Counting number of logged-in and non-logged-in visitors based on their source

``logged_in_visitors(visitors)`` determines whether a visitor is logged in based on their visitor UUID. It iterates through the list of visitors and verifies the user's logged-in status by checking for the presence of the ``visitor_username`` field.

``non_logged_in(visitors)`` works like ``logged_in_visitors(visitors)`` but inverse, it focuses on entries that specifically do not have the ``visitor_username`` field.

Both functions return the count of their respective values, grouped by source derived from the ``visitor_source`` field. The aforementioned functions are invoked together to provide a comparative study between logged-in and non-logged-in users, as well as what source they come from. [\[Code Snippet 5\]](#)

5.6.10. Identifying Authenticated Visitors

``visitor_authenticated(visitor_uuid, visitors)`` function determines whether a visitor is logged in based on their visitor UUID. It iterates through the list of visitors and verifies the user's logged-in status by

checking for the presence of the `visitor\_username` field. The function prints the user's logged-in status along with the visitor username (if the user is logged in).

5.7 Graphs and GUI

The `graphs.py` module is responsible for generating histograms, bar graphs (**Additional Feature**) and pie charts (**Additional Feature**). This file contains functions for visualizing country-wise views, continent-wise views, browser-wise views, document overviews and visitor overviews. Additionally, it features a function for generating a graph showing the relationship between visitors and documents based on the `also\_likes` functionality. **Matplotlib** and **Graphviz** libraries are used for graph plotting and visualization purposes.

`gui.py` is wrapped into a module that encapsulates the code for GUI functionality. It is developed using **Tkinter** for GUI components and **Matplotlib** for interactive graph plotting.

6. Testing

To assess the core functionalities of the application, unit testing was conducted using the **Pytest** framework. For testing purposes, users should navigate to the root directory `F20SC-CW2` and execute the command `pytest` in the terminal. Alternatively, if the testing has been set up, users can access the `unit\_test.py` file and run the tests.

6.1 Invalid Input

T1: Invalid File Path

Testing if an exception is raised when the user enters an invalid file path.

Expected Result	Result
Exception	Success

T2 : Invalid Document UUID

Testing if an exception is raised when the user enters an invalid document UUID.

Expected Result	Result
Exception	Success

T3 : Invalid Visitor UUID

Testing if an exception is raised when the user enters an invalid visitor UUID.

Expected Result	Result
Exception	Success

6.2 Authenticating Inputs

T4 : Validates Document UUID

Asserts true when the user enters a valid Document UUID.

Expected Result	Result
Asserts True	Success

T5 : Validates Visitor UUID

Asserts true when the user enters a valid visitor UUID.

Expected Result	Result
Asserts True	Success

6.3 Verifying Country and Continent Count

T6: Verifying Country Count

For the specified document UUID, this unit test validates if the output is **'Argentina'**.

Expected Result	Result
Asserting Output	Success

T7: Verifying Continent Count

For the specified document UUID, this unit test validates if the output is **'South America'**.

Expected Result	Result
Asserting Output	Success

6.4 Avid Reader Functionality

T8: Testing `avid\_readers()` functionality

Asserts true when the output of **`avid\_readers(visitors)`** function is printed in descending order.

Expected Result	Result
Asserts True	Success

6.5 `Also\_likes` Functionality

T9 : Testing `also\_likes` functionality

The *"Also Likes"* functionality involves the creation of a system that identifies and presents documents related to a given document based on shared readership. In this scenario, we test a specific document UUID linked to multiple associated UUIDs. The **`also\_likes`** function is evaluated by verifying that its output contains more than one document UUID.

Expected Result	Result
-----------------	--------

Asserting Output

Success

7. What did I learn from CW1?

7.1 Code Feedback

In response to the feedback about using a single file and not utilizing object-oriented programming principles in the previous coursework, for this coursework, we structured the code to promote object-oriented programming using classes, functions, and custom-built modules. The project wraps functions related to core requirements, additional functionalities, and GUI into modules enhancing accessibility.

7.2 Report Feedback

In the previous coursework, it was pointed out that the design consideration was brief. To address this feedback in coursework 2, the design consideration section has been expanded to provide a comprehensive insight into the code structure and the data structures used in the code. The section also discusses the scalability of the code and its robustness to run-time errors and corner cases.

8. Conclusions

In conclusion, we are satisfied with the overall implementation of the program. Our code structure adheres to the principles of Object-Oriented Programming (OOP). We were able to achieve all the core requirements mentioned in the coursework specification, and successfully implement additional features. The application has undergone extensive unit testing to ensure it is bug-free. The user-friendly GUI offers a smooth platform for users to interact with the application seamlessly.

8.1 Future Works

Larger datasets such as the 3-million-line dataset require a longer execution time. For future improvements, we will implement threading to enhance processing speed and overall user experience.

8.2 Comparison of Software Development in Scripting vs System Language

In Coursework 1, we constructed a C#-based web browser, engaging in low-level programming for enhanced control over system resources. Although C# allowed for efficient memory management and direct system functionality access, the manual memory handling and resource management added complexity and time to the development process. Coursework 2 focused on a Python-based data analysis tool, leveraging the language's high-level nature and extensive libraries for quicker development and simplification of complex tasks. However, Python's automatic memory management and abstraction from system-level details meant less control over system resources compared to C#. Ultimately, the choice between system and scripting languages depends on project requirements, favouring C# for performance-critical applications and Python for scenarios prioritizing rapid development and ease of maintenance.

9. References

1. <https://martinxpn.medium.com/what-are-packages-in-python-and-what-is-the-role-ofinit-py-files-82-100-days-of-python-325a992b2b13>
2. <https://pypi.org/project/pyconvert/>
3. <https://stackoverflow.com/questions/27627440/python-user-agents-library-is-not-working>
4. <https://pypi.org/project/ua-parser/>

10. Appendix

10.1 Contributions

- Archisa Kar: GUI | Graphs | Code Documentation
- Asmitha Krishnakumar: Backend | Code Documentation | Report Writing
- Suhail Ahmed Basheer: Team Lead | Full-Stack | Code Documentation

The number of viewers from each continent for the document:

Document UUID:

Plot Histogram

Plot Bar Graph

Plot Pie Chart

Back to Home

New tab

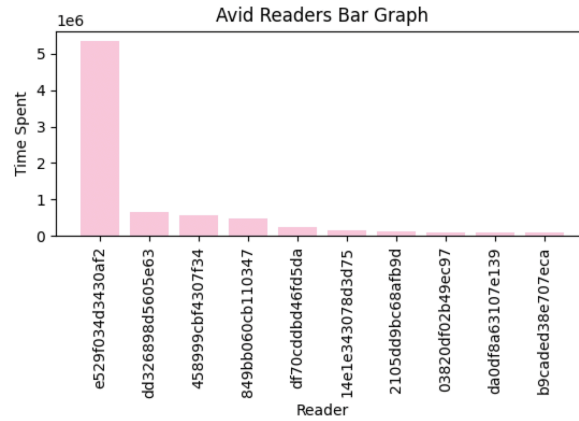
The browsers used to access the documents:

Plot Histogram

Plot Bar Graph

Back to Home

New tab for different browsers.



Top 10 avid readers:

Reader 1's UUID: e529f034d3430af2
Reading Time - 5356278

Reader 2's UUID: dd326898d5605e63
Reading Time - 648318

Reader 3's UUID: 458999cbf4307f34
Reading Time - 576177

Reader 4's UUID: 849bb060cb110347
Reading Time - 486053

Reader 5's UUID: df70cddbd46fd5da
Reading Time - 239350

Reader 6's UUID: 14e1e343078d3d75
Reading Time - 161381

Reader 7's UUID: 2105dd9bc68afb9d
Reading Time - 109458

Reader 8's UUID: 03820df02b49ec97
Reading Time - 102493

Reader 9's UUID: da0df8a63107e139
Reading Time - 98291

Reader 10's UUID: b9caded38e707eca
Reading Time - 98183

Output for top 10 avid readers.

Also Likes Graph

Document UUID:

Visitor UUID: (Optional)

Sort Order: (Optional)

None
Ascending
Descending

Also Likes Graph

Back to Home

New tab for also likes graph.

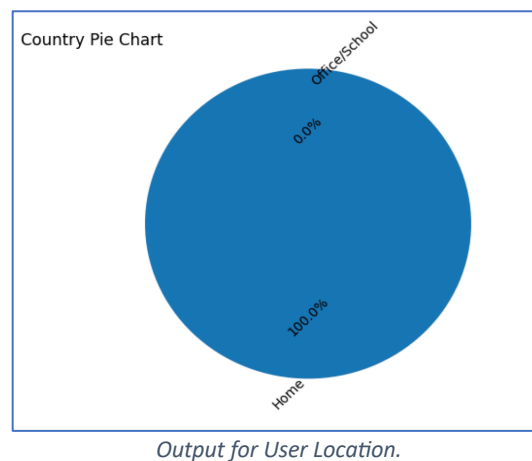
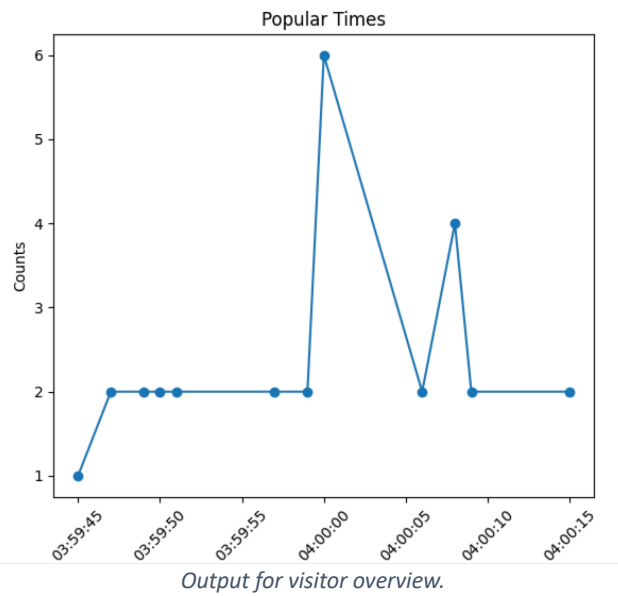
Most Popular for a visitor to view documents

Visitor UUID:

Plot

Back to Home

New tab for entering Visitor UUID.



Check is the visitor is logged in or not

Visitor UUID:

98ac6a1ca9476771

Visitor UUID: 98ac6a1ca9476771

The visitor is not logged in.

Check Visitor

Back to Home

Output for User Authentication

```

def also_likes(documents, doc_uuid, visitor_uuid=None, sorting_func=None):
    # Dictionary to store the relationship between visitors and documents
    visitor_doc_relationship = {}
    # Counter to store the occurrences of each visitor
    visitor_counter = {}
    # Counter to store the occurrences of each document
    doc_counter = {}
    # Dictionary to store the mapping between visitors and the documents they have read
    doc_visitor_mapping = {}

    # Get the list of visitors for the specified document UUID
    visitors = doc_to_visitor(documents, doc_uuid)

    # Build a mapping between visitors and the documents they have visited
    for visitor in visitors:
        if visitor not in visitor_doc_relationship:
            visitor_doc_relationship[visitor] = visitor_to_doc(documents, visitor)

    # Count the number of readers for each document
    for visitor in visitor_doc_relationship:
        for document in visitor_doc_relationship[visitor]:
            # Increment the count for the document
            if document in doc_counter:
                doc_counter[document] += 1
            else:
                doc_counter[document] = 1

    # Sort the documents based on the specified sorting function
    doc_counter = sortingfunc_test(doc_counter, sorting_func)
    # Select the top 7 documents based on the sorting result
    doc_counter = {key: doc_counter[key] for key in list(doc_counter)[:7]}

    # Count the occurrences of each visitor for the top documents
    for document in doc_counter:
        document_visitors = doc_to_visitor(documents, document)
        for visitor in document_visitors:
            # Increment the count for the visitor
            if visitor not in visitor_counter:
                visitor_counter[visitor] = 1
            else:
                visitor_counter[visitor] += 1

    # Create a copy of visitor_counter for further processing
    visitor_top_counter = visitor_counter.copy()

    # Sort the visitors based on the specified sorting function
    visitor_top_counter = sortingfunc_test(visitor_top_counter, sorting_func)
    # Select the top 3 visitors based on the sorting result
    visitor_top_counter = {key: visitor_top_counter[key] for key in list(visitor_top_counter)[:3]}

    # If a specific visitor UUID is provided and not in the top visitors, add it with its count
    if visitor_uuid is not None and visitor_uuid not in visitor_top_counter:
        if visitor_uuid in visitor_counter:
            visitor_top_counter[visitor_uuid] = visitor_counter[visitor_uuid]
        else:
            visitor_top_counter[visitor_uuid] = 0

    # Create a mapping between visitors and the documents they have read among the top documents
    for visitor in visitor_top_counter:
        visited_documents = visitor_to_doc(documents, visitor)

        # Remove documents that are not among the top documents
        for document in visited_documents.copy():
            if document not in doc_counter:
                visited_documents.remove(document)

        # Store the mapping in doc_visitor_mapping
        doc_visitor_mapping[visitor] = visited_documents

    # Return the results
    return doc_counter, visitor_top_counter, doc_visitor_mapping

```

Code snippet for also_likes()

```

1 def sortingfunc_test(doc_reader_count, reverse):
2
3     if type(doc_reader_count) is dict:
4         # Sort the document-reader count dictionary based on the count
5         return dict(sorted(doc_reader_count.items(), key=lambda item: item[1], reverse=reverse))
6     else:
7         # Sort the list of tuples based on the count
8         return sorted(doc_reader_count.items(), key=lambda item: item[1], reverse=reverse)

```

Code snippet for sorting function()

```

1 def validation(doc_uuid=None, visitor_uuid=None, documents=None, visitors=None):
2     valid = True
3     # Check if the document UUID is valid
4     if doc_uuid is not None and doc_uuid is not None:
5         if doc_uuid not in documents:
6             valid = "document"
7
8     # Check if the visitor UUID is valid
9     if visitor_uuid is not None and visitors is not None:
10        if visitor_uuid not in visitors:
11            valid = "visitor"
12
13    if valid is not True:
14        raise Exception("Invalid {x} UUID. Please try again.".format(x=valid))
15    else:
16        return valid

```

Code snippet for validation()

```

1 def ip_to_location(documents, ip_address=None):
2     # Dictionary to store count of home and office/school views
3     views = {"Home": 0, "Office/School": 0}
4
5     # List to store unique hours (for non-specified IP addresses)
6     l = []
7
8     # Iterate through each document UUID in the documents dictionary
9     for doc_uuid in documents:
10        # Iterate through each event for the current document UUID
11        for event in documents[doc_uuid]:
12
13            # If a specific IP address is provided, check for its presence in the event
14            if ip_address is not None:
15                if "visitor_ip" in event and event["visitor_ip"] == ip_address:
16                    time = datetime.fromtimestamp(int(event.get("ts")))
17
18            # If no specific IP address is provided, consider all events and track hours for non-specified IP addresses
19            else:
20                time = datetime.fromtimestamp(int(event.get("ts")))
21                if time.hour not in l:
22                    l.append(time.hour)
23
24            # Check if the event timestamp falls within office/school hours (9am to 5pm)
25            if time.hour >= 9 and time.hour <= 17:
26                views["Office/School"] += 1
27            else:
28                views["Home"] += 1
29
30    return views

```

Code snippet for ip_to_location()

```
def logged_in_visitors(visitors):
    logged_in_visitors = {}

    for visitor in visitors:
        for entries in visitors[visitor]:
            # Check for the presence of both "visitor_username" and "visitor_source" in the event
            if "visitor_username" in list(entries.keys()) and "visitor_source" in list(entries.keys()):
                # Update the count of logged-in visitors for each source
                if entries["visitor_source"] in list(logged_in_visitors.keys()):
                    logged_in_visitors[entries["visitor_source"]] += 1
                else:
                    logged_in_visitors[entries["visitor_source"]] = 1

    return logged_in_visitors

def non_logged_in_visitors(visitors):
    non_logged_in_visitors = {}

    for visitor in visitors:
        for entries in visitors[visitor]:
            # Check for the absence of "visitor_username" and the presence of "visitor_source" in the event
            if "visitor_username" not in list(entries.keys()) and "visitor_source" in list(entries.keys()):
                # Update the count of non-logged-in visitors for each source
                if entries["visitor_source"] in list(non_logged_in_visitors.keys()):
                    non_logged_in_visitors[entries["visitor_source"]] += 1
                else:
                    non_logged_in_visitors[entries["visitor_source"]] = 1

    return non_logged_in_visitors
```

Code snippet for logged in and non-logged in visitors